LAPORAN PRAKTIKUM

DEEP LEARNING

GATED RECURRENT UNIT (GRU)
UNTUK PREDIKSI DATA SEKUENSIAL

Disusun oleh:

HABIBI PUTRA RIZQULLAH

2411531001

Praktikum Kelas B

DOSEN PENGAMPU:

Dr. Derisma, S.T., M.T.

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

DAFTAR ISI

BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 1
1.3 Tujuan Praktikum 2
1.4 Manfaat Praktikum 2
BAB II LANDASAN TEORI 3
2.1 Data Sekuensial dan Pembentukan Sequence 3
2.2 Gated Recurrent Unit 3
2.3 Mekanisme Gerbang GRU 3
2.4 Perbandingan GRU dan LSTM 3
2.5 MSE dan Optimizer Adam 3
2.6 Hidden Size dan Jumlah Layer 4
2.7 Batas Evaluasi pada Praktikum 4
2.8 Aliran Data pada Satu Sequence 4
2.9 Bentuk Tensor dan Keluaran Model 5
2.10 Perhitungan Jumlah Parameter 6
BAB III METODOLOGI DAN LANGKAH PRAKTIKUM 7
3.1 Alat dan Bahan 7
3.2 Alur Pengerjaan 7
3.3 Praktikum Prediksi Deret Sinus 7
3.3.1 Persiapan Library 7
3.3.2 Pembuatan Dataset dan Sequence 8
3.3.3 Definisi Model GRU 8
3.3.4 Inisialisasi Model dan Optimizer 8
3.3.5 Training Model 9
3.3.6 Visualisasi Loss 9
3.3.7 Prediksi Deret Sinus 10
3.3.8 Pengukuran Waktu dan Pembanding LSTM 11
3.4 Tugas Praktikum Prediksi Deret Cosinus 11
3.4.1 Pembuatan Data Cosinus 11
3.4.2 Konfigurasi dan Pelatihan Model 12
3.4.3 Penyusunan Tabel Hasil 13
3.4.4 Visualisasi Perbandingan 13
3.5 Pemeriksaan Pembentukan Data 15
3.6 Urutan Pembaruan Bobot dan Pencatatan Loss 16
BAB IV HASIL DAN PEMBAHASAN 17
4.1 Hasil Praktikum Deret Sinus 17

4.2 Hasil Tugas Praktikum Deret Cosinus 17
4.2.1 Pengaruh Hidden Size 18
4.2.2 Pengaruh Penambahan Layer 18
4.3 Perbandingan GRU dan LSTM 18
4.4 Analisis Kurva Loss dan Prediksi 19
4.5 Jawaban Pertanyaan Reflektif 20
4.5.1 Mengapa GRU Lebih Cepat daripada LSTM 20
4.5.2 Peran Update Gate dalam GRU 20
4.5.3 Penggunaan GRU untuk Teks yang Sangat Panjang 20
4.6 Selisih Waktu Training Antarmodel 21
4.7 Pemilihan Konfigurasi Berdasarkan Hasil 22
BAB V PENUTUP 23
5.1 Kesimpulan 23
DAFTAR PUSTAKA 24

DAFTAR GAMBAR

Gambar 3.1 Screenshot import library 7
Gambar 3.2 Screenshot pembuatan data sinus dan sequence 8
Gambar 3.3 Screenshot definisi model GRU 8
Gambar 3.4 Screenshot inisialisasi model loss dan optimizer 8
Gambar 3.5 Screenshot loop training GRU 9
Gambar 3.6 Screenshot output training GRU 9
Gambar 3.7 Screenshot kode visualisasi loss 9
Gambar 3.8 Output grafik training loss GRU 10
Gambar 3.9 Screenshot kode prediksi data sinus 10
Gambar 3.10 Output prediksi deret sinus 11
Gambar 3.11 Screenshot output perbandingan pada data sinus 11
Gambar 3.12 Screenshot kode dataset cosinus 12
Gambar 3.13 Output visualisasi data cosinus 12
Gambar 3.14 Screenshot kode eksperimen GRU dan LSTM 12
Gambar 3.15 Screenshot output tabel hasil eksperimen 13
Gambar 3.16 Screenshot kode grafik perbandingan 13
Gambar 3.17 Output perbandingan hidden size 32 dan satu layer 14
Gambar 3.18 Output perbandingan hidden size 64 dan satu layer 14
Gambar 3.19 Output perbandingan hidden size 64 dan dua layer 14

DAFTAR TABEL

Tabel 2.1 Susunan jendela input dan target 5
Tabel 2.2 Perubahan bentuk tensor pada model dasar 5
Tabel 2.3 Jumlah parameter berdasarkan konfigurasi 6
Tabel 3.1 Pemeriksaan susunan dataset 15
Tabel 3.2 Urutan operasi dalam satu epoch 16
Tabel 4.1 Hasil perbandingan GRU dan LSTM pada data sinus 17
Tabel 4.2 Hasil tiga konfigurasi GRU pada data cosinus 17
Tabel 4.3 Perbandingan GRU dan LSTM pada data cosinus 18
Tabel 4.4 Selisih waktu GRU terhadap LSTM 21
Tabel 4.5 Pertimbangan konfigurasi GRU 22

BAB I
PENDAHULUAN

1.1 Latar Belakang

Data sekuensial memiliki urutan yang perlu diperhatikan saat diolah. Pada deret waktu, nilai sebelumnya dapat membantu memperkirakan nilai berikutnya. Model recurrent memproses urutan tersebut melalui hidden state yang diperbarui pada setiap langkah.

Praktikum sebelumnya menggunakan LSTM untuk memprediksi data sinus dan cosinus. Pada praktikum ini digunakan Gated Recurrent Unit atau GRU. GRU mengatur informasi melalui update gate dan reset gate, dengan susunan yang lebih sederhana daripada LSTM [1], [2]. Perbedaannya diamati melalui loss, kurva prediksi, dan waktu training.

Percobaan dimulai dari kode GRU pada modul menggunakan data sinus. Pada tugas individu, data diganti menjadi cosinus dan model dilatih dengan tiga konfigurasi hidden size serta jumlah layer. LSTM dari Praktikum 4 dilatih ulang menggunakan data dan pengaturan yang sama agar hasilnya dapat dibandingkan.

1.2 Rumusan Masalah

1. Bagaimana membentuk data sinus dan cosinus menjadi sequence untuk model GRU?
2. Bagaimana menerapkan GRU menggunakan PyTorch untuk memprediksi nilai berikutnya?
3. Bagaimana pengaruh hidden size dan jumlah layer terhadap loss serta waktu training?
4. Bagaimana perbandingan hasil GRU dan LSTM pada konfigurasi yang sama?

1.3 Tujuan Praktikum

1. Membentuk data sinus dan cosinus menjadi sequence sebagai input model GRU.
2. Menerapkan model GRU menggunakan PyTorch untuk memprediksi nilai berikutnya.
3. Menganalisis pengaruh hidden size dan jumlah layer terhadap loss serta waktu training.
4. Membandingkan loss akhir dan waktu training GRU dengan LSTM pada konfigurasi yang sama.

1.4 Manfaat Praktikum

Melalui data sinus dan cosinus, hasil prediksi dapat diamati langsung pada grafik. Perubahan konfigurasi membantu memahami hubungan antara ukuran model, proses training, dan kesalahan prediksi tanpa perlu menyiapkan dataset yang rumit.

BAB II
LANDASAN TEORI

2.1 Data Sekuensial dan Pembentukan Sequence

Data sekuensial disusun menurut urutan tertentu. Pada praktikum ini, deret sepanjang 1.000 nilai dibentuk menjadi pasangan input dan target menggunakan sliding window. Sebanyak 20 nilai berurutan menjadi input, sedangkan nilai sesudahnya menjadi target. Proses ini menghasilkan 980 pasangan data.

2.2 Gated Recurrent Unit

GRU merupakan model recurrent dengan dua gerbang utama. Update gate mengatur berapa banyak informasi lama yang dipertahankan saat hidden state diperbarui. Reset gate mengatur pengaruh hidden state sebelumnya terhadap kandidat informasi baru [1], [2]. GRU tidak memiliki cell state yang terpisah seperti LSTM.

2.3 Mekanisme Gerbang GRU

Pada implementasi PyTorch, reset gate dan update gate dihitung menggunakan fungsi sigmoid. Kandidat hidden state dihitung menggunakan tanh. Hidden state berikutnya merupakan gabungan antara kandidat baru dan hidden state sebelumnya, dengan bobot yang diatur oleh update gate [2]. Nilai gerbang berada di antara 0 dan 1 sehingga model dapat mengatur informasi secara bertahap.

2.4 Perbandingan GRU dan LSTM

LSTM menggunakan input gate, forget gate, dan output gate serta cell state terpisah [3]. GRU memakai dua gerbang dan langsung memperbarui hidden state. Pada input size, hidden size, dan jumlah layer yang sama, GRU memiliki lebih sedikit parameter. Walaupun demikian, model yang lebih sederhana tidak otomatis memberikan loss terendah atau waktu tercepat pada semua perangkat.

2.5 MSE dan Optimizer Adam

Mean Squared Error atau MSE menghitung rata-rata kuadrat selisih nilai prediksi dan target. Karena keluarannya berupa angka kontinu, praktikum ini menggunakan MSE sebagai loss. Bobot diperbarui menggunakan Adam dengan learning rate 0,01. Adam menyesuaikan pembaruan parameter berdasarkan perkiraan momen pertama dan kedua gradien [4].

2.6 Hidden Size dan Jumlah Layer

Hidden size menentukan ukuran representasi yang disimpan pada hidden state. Jumlah layer menentukan banyaknya lapisan recurrent yang ditumpuk. Penambahan keduanya meningkatkan jumlah parameter dan kapasitas model, tetapi hasil akhirnya tetap dipengaruhi data, inisialisasi bobot, learning rate, dan jumlah epoch.

2.7 Batas Evaluasi pada Praktikum

Kode modul menggunakan seluruh sequence untuk training dan prediksi. Oleh karena itu, loss yang dilaporkan menunjukkan kesalahan pada data yang telah dipelajari. Grafik yang mengikuti data asli menunjukkan kecocokan pada data tersebut, tetapi belum membuktikan kemampuan model pada data baru. Prediksi dilakukan satu langkah ke depan dengan riwayat aktual, bukan dengan memasukkan hasil prediksi secara berulang.

2.8 Aliran Data pada Satu Sequence

Satu sampel masukan berisi 20 angka, bukan satu angka saja. Pada sampel pertama, model membaca data indeks 0 sampai 19 secara berurutan. Targetnya adalah data indeks 20. Sampel kedua bergeser satu posisi sehingga inputnya menjadi indeks 1 sampai 20 dan targetnya indeks 21. Pergeseran ini diteruskan sampai target terakhir pada indeks 999.

Hidden state diperbarui pada setiap langkah di dalam satu sampel. Pada langkah pertama, model menerima satu nilai dan hidden state awal nol. Pada langkah berikutnya, model menerima nilai selanjutnya beserta hidden state yang baru dihitung. Setelah 20 langkah, representasi terakhir diteruskan ke lapisan linear. Dengan demikian, satu sequence menghasilkan satu prediksi.

Pada kode ini, hidden state awal tidak diberikan secara manual dan tidak diteruskan dari satu pemanggilan model ke pemanggilan berikutnya. PyTorch menggunakan hidden state awal nol. Walaupun jendela data saling bertumpang tindih, setiap sampel tetap diproses sebagai sequence tersendiri dalam batch.

Prediksi pertama diletakkan pada indeks 20. Oleh karena itu, bagian awal grafik hanya memiliki garis data asli. Ketiadaan garis prediksi pada dua puluh titik awal bukan berarti model gagal menghasilkan keluaran, tetapi karena titik tersebut dipakai untuk membentuk input pertama.

Sampel	Indeks input	Indeks target
Pertama	0 sampai 19	20
Kedua	1 sampai 20	21
Terakhir	979 sampai 998	999

Tabel 2.1 Susunan jendela input dan target

2.9 Bentuk Tensor dan Keluaran Model

Setelah create_sequences dijalankan, array input memiliki 980 baris dan 20 kolom. Tiap baris menyimpan satu jendela waktu. Pemanggilan unsqueeze(-1) menambahkan dimensi fitur sehingga bentuk tensor menjadi (980, 20, 1). Nilai 1 menunjukkan bahwa setiap langkah hanya memiliki satu fitur.

Parameter batch_first=True membuat urutan dimensi menjadi jumlah sampel, panjang sequence, dan jumlah fitur. Pada model dasar dengan hidden size 64, keluaran GRU memiliki bentuk (980, 20, 64). Artinya, tersedia representasi berukuran 64 untuk setiap langkah waktu pada setiap sampel.

Ekspresi out[:, -1, :] memilih seluruh sampel, mengambil langkah waktu terakhir, dan mempertahankan seluruh komponen hidden. Hasilnya berbentuk (980, 64). Lapisan nn.Linear(64, 1) kemudian mengubah representasi tersebut menjadi (980, 1), sama seperti bentuk target y.

Jumlah layer tidak mengubah panjang sequence atau jumlah keluaran akhir. Ketika dua layer dipakai, layer kedua menerima representasi dari layer pertama pada setiap langkah waktu. Keluaran yang diambil tetap berasal dari langkah terakhir pada layer paling atas.

Tahap	Bentuk tensor	Arti dimensi terakhir
Input X	(980, 20, 1)	Satu fitur
Output GRU	(980, 20, 64)	Hidden size 64
Langkah terakhir	(980, 64)	Hidden size 64
Output linear	(980, 1)	Satu prediksi
Target y	(980, 1)	Satu nilai aktual

Tabel 2.2 Perubahan bentuk tensor pada model dasar

2.10 Perhitungan Jumlah Parameter

Jumlah parameter dapat dihitung dari ukuran input dan hidden state. Pada satu layer GRU dengan bias aktif, tiga kelompok perhitungan digunakan untuk reset gate, update gate, dan kandidat hidden state. Jika ukuran input dinyatakan sebagai I dan hidden size sebagai H, jumlah parameter recurrent adalah 3H(I + H + 2) [2].

Komponen 3HI berasal dari bobot input, sedangkan 3HH berasal dari bobot hidden. Dua kelompok bias masing-masing berukuran 3H sehingga total biasnya 6H. Lapisan linear dengan satu output menambah H bobot dan satu bias. Untuk model satu layer, jumlah keseluruhannya adalah 3H(I + H + 2) + H + 1.

Pada model dasar, I = 1 dan H = 64. Jumlah parameter GRU adalah 3 x 64 x (1 + 64 + 2) + 64 + 1 = 12.929. Dengan konfigurasi yang sama, LSTM memakai empat kelompok perhitungan sehingga jumlahnya menjadi 4 x 64 x (1 + 64 + 2) + 64 + 1 = 17.217 [3].

Perhitungan ini menjelaskan mengapa GRU memiliki ukuran model yang lebih kecil. Namun, jumlah parameter bukan satu-satunya penentu waktu eksekusi. Implementasi operasi recurrent, jumlah thread, dan kondisi komputer juga memengaruhi pengukuran. Angka parameter berikut dihitung dari arsitektur, bukan hasil training tambahan.

Konfigurasi	Parameter GRU	Parameter LSTM
Hidden 32, 1 layer	3.393	4.513
Hidden 64, 1 layer	12.929	17.217
Hidden 64, 2 layer	37.889	50.497

Tabel 2.3 Jumlah parameter berdasarkan konfigurasi

BAB III
METODOLOGI DAN LANGKAH PRAKTIKUM

3.1 Alat dan Bahan

Praktikum dikerjakan menggunakan Google Colaboratory dan Python. PyTorch digunakan untuk membangun model, NumPy untuk membuat data, Matplotlib untuk menggambar grafik, Pandas untuk menyusun tabel, serta modul time untuk mengukur waktu training. Berkas kerja yang digunakan adalah 2411531001_Habibi_Putra_Rizqullah_Program5.ipynb.

Kode tidak memindahkan tensor atau model ke CUDA. Dengan demikian, proses komputasi pada kode ini menggunakan CPU meskipun lingkungan Colab dapat menyediakan GPU. Versi library dan jenis CPU tidak dicatat pada output notebook.

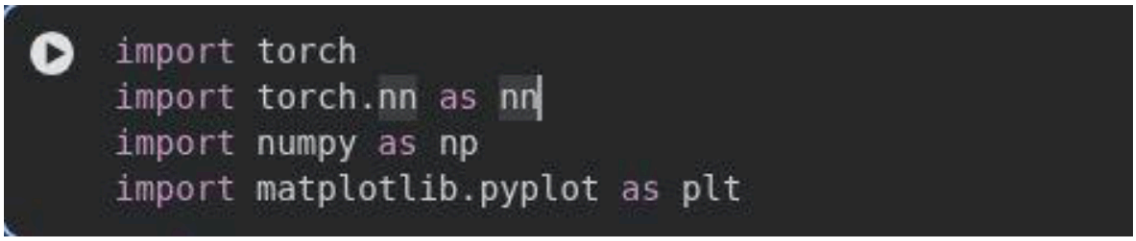
3.2 Alur Pengerjaan

Data sinus dibuat sebanyak 1.000 titik pada rentang 0 sampai 100. Data dibentuk menjadi sequence sepanjang 20, kemudian model GRU dilatih selama 50 epoch. Setelah grafik loss dan prediksi ditampilkan, GRU serta LSTM dilatih ulang untuk mencatat loss dan waktu training. Pada tugas individu, data diganti menjadi cosinus dan masing-masing arsitektur dijalankan pada tiga konfigurasi.

3.3 Praktikum Prediksi Deret Sinus

3.3.1 Persiapan Library

Library torch dan torch.nn digunakan untuk model serta proses training. NumPy digunakan untuk membuat nilai sinus, sedangkan Matplotlib digunakan untuk visualisasi. Import pada bagian ini mengikuti kode modul.



Gambar 3.1 Screenshot import library

3.3.2 Pembuatan Dataset dan Sequence

Data dibuat dengan np.sin(np.linspace(0, 100, 1000)). Fungsi create_sequences mengambil 20 nilai sebagai input dan satu nilai berikutnya sebagai target. Setelah diubah menjadi tensor, X memiliki bentuk (980, 20, 1), sedangkan y memiliki bentuk (980, 1). Tidak ada pembagian training dan testing.

```python
# 1. Membuat dataset sinusoidal
data = np.sin(np.linspace(0, 100, 1000))
seq_length = 20

def create_sequences(data, seq_length):
    xs, ys = [], []
    for i in range(len(data) - seq_length):
        xs.append(data[i:i+seq_length])
        ys.append(data[i+seq_length])
    return np.array(xs), np.array(ys)

X, y = create_sequences(data, seq_length)
X = torch.FloatTensor(X).unsqueeze(-1)
y = torch.FloatTensor(y).unsqueeze(-1)
```

Gambar 3.2 Screenshot pembuatan data sinus dan sequence

3.3.3 Definisi Model GRU

GRUModel terdiri dari nn.GRU dan nn.Linear. Parameter batch_first=True membuat dimensi pertama input menjadi dimensi sampel. Keluaran pada langkah terakhir, out[:, -1, :], diteruskan ke lapisan linear untuk menghasilkan satu nilai prediksi.

```python
# 2. Definisi model GRU
class GRUModel(nn.Module):
    def __init__(self, input_size=1, hidden_size=64, output_size=1, num_layers=1):
        super(GRUModel, self).__init__()
        self.gru = nn.GRU(input_size, hidden_size, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        out, _ = self.gru(x)
        out = self.fc(out[:, -1, :])
        return out
```

Gambar 3.3 Screenshot definisi model GRU

3.3.4 Inisialisasi Model dan Optimizer

Model dasar menggunakan hidden size 64 dan satu layer. Fungsi loss ditetapkan sebagai nn.MSELoss(), sedangkan optimizer menggunakan torch.optim.Adam dengan learning rate 0,01.

```python
# 3. Inisialisasi model, loss, optimizer
model = GRUModel()
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

Gambar 3.4 Screenshot inisialisasi model loss dan optimizer

3.3.5 Training Model

Setiap epoch dimulai dengan mengosongkan gradien. Model menghasilkan prediksi, loss dihitung, kemudian backpropagation dijalankan melalui loss.backward(). Bobot diperbarui dengan optimizer.step(). Loss disimpan pada setiap epoch dan dicetak setiap 10 epoch.

```
# 4. Training loop
epochs = 50
losses = []
for epoch in range(epochs):
    optimizer.zero_grad()
    outputs = model(X)
    loss = criterion(outputs, y)
    loss.backward()
    optimizer.step()
    losses.append(loss.item())
    if (epoch+1) % 10 == 0:
        print(f'Epoch [{epoch+1}/{epochs}], Loss: {loss.item():.4f}')
```

Gambar 3.5 Screenshot loop training GRU

```
Epoch [10/50], Loss: 0.0326
Epoch [20/50], Loss: 0.0092
Epoch [30/50], Loss: 0.0016
Epoch [40/50], Loss: 0.0003
Epoch [50/50], Loss: 0.0003
```

Gambar 3.6 Screenshot output training GRU

Output menunjukkan loss sebesar 0,0326 pada epoch ke-10, turun menjadi 0,0092 pada epoch ke-20 dan 0,0016 pada epoch ke-30. Pada epoch ke-40 dan ke-50, loss yang ditampilkan telah mencapai 0,0003. Angka tersebut mengikuti pembulatan empat desimal pada print program.

3.3.6 Visualisasi Loss

Daftar losses digambar untuk melihat perubahan kesalahan selama training. Secara umum, loss menurun dan mendekati nol pada bagian akhir pelatihan.

```
# 5. Plot loss
plt.plot(losses)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss GRU')
plt.show()
```

Gambar 3.7 Screenshot kode visualisasi loss



Gambar 3.8 Output grafik training loss GRU

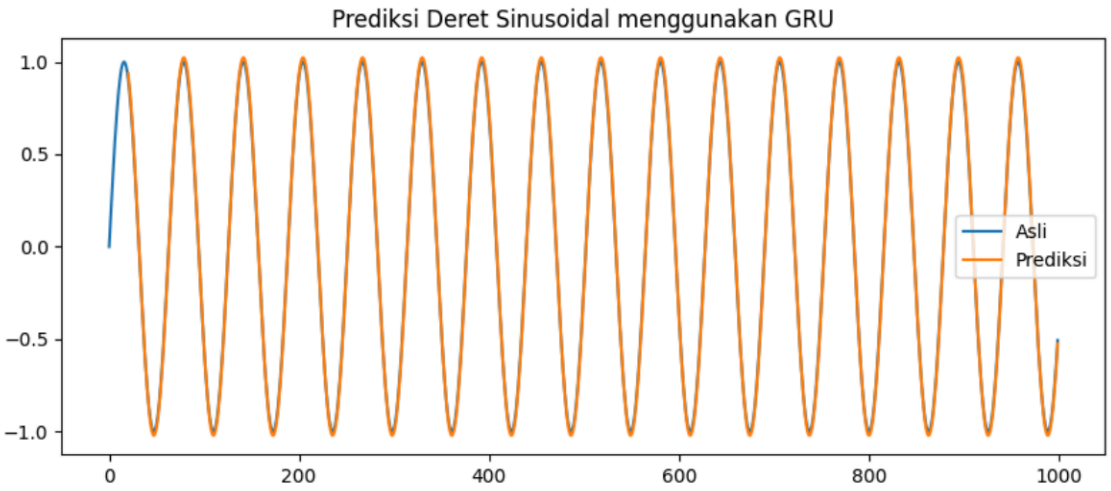
3.3.7 Prediksi Deret Sinus

Prediksi dihitung dengan torch.no_grad() agar gradien tidak dihitung. Hasilnya digambar mulai indeks ke-20 karena dua puluh nilai pertama digunakan sebagai input awal. Garis prediksi mengikuti pola sinus dan terlihat dekat dengan data asli.

```
# 6. Prediksi
with torch.no_grad():
    preds = model(X).numpy()

plt.figure(figsize=(10,4))
plt.plot(data, label='Asli')
plt.plot(np.arange(seq_length, len(preds)+seq_length), preds, label='Prediksi')
plt.legend()
plt.title('Prediksi Deret Sinusoidal menggunakan GRU')
plt.show()
```

Gambar 3.9 Screenshot kode prediksi data sinus



Gambar 3.10 Output prediksi deret sinus

3.3.8 Pengukuran Waktu dan Pembanding LSTM

LSTMModel dari Praktikum 4 digunakan kembali sebagai pembanding. Fungsi jalankan_eksperimen melatih setiap model selama 50 epoch dan mengukur waktu loop training menggunakan time.perf_counter(). Seed 42 diatur sebelum model dibuat. Setelah training selesai, fungsi mengembalikan model, riwayat loss, prediksi, dan waktu training.

Pengukuran dilakukan ulang pada GRU dan LSTM dengan hidden size 64 serta satu layer. Loss pada tabel diambil dari losses[-1], yaitu loss epoch terakhir sebelum pembaruan bobot terakhir. Waktu tidak mencakup inisialisasi model, pembuatan grafik, atau perhitungan prediksi setelah training.

	Model	Hidden Size	Num Layers	Loss Akhir	Waktu Training (s)
0	GRU	64	1	0.000070	3.692325
1	LSTM	64	1	0.000403	7.330393

Gambar 3.11 Screenshot output perbandingan pada data sinus

3.4 Tugas Praktikum Prediksi Deret Cosinus

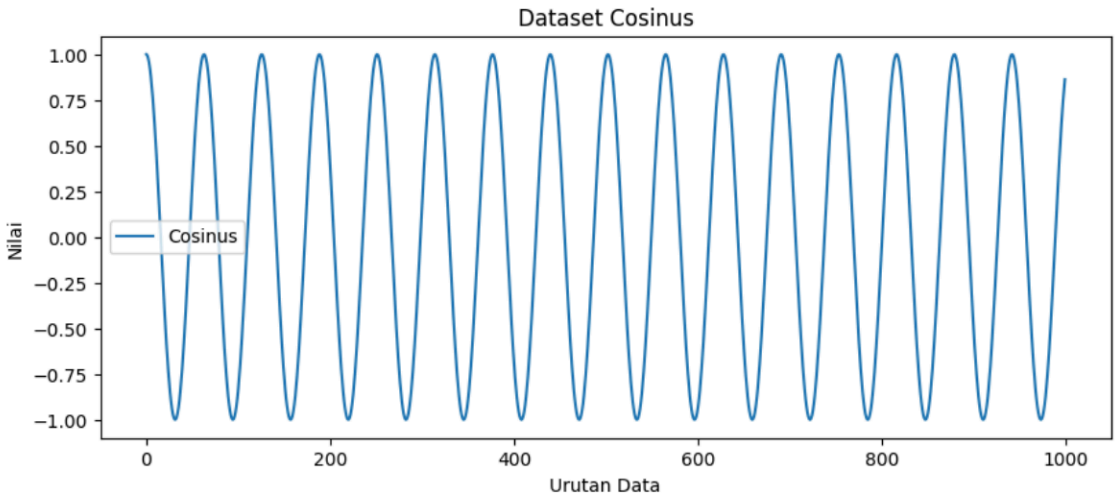
3.4.1 Pembuatan Data Cosinus

Fungsi pembangkit data diganti dari sinus menjadi cosinus. Jumlah titik, rentang, panjang sequence, serta pembentukan tensor tetap sama. Seluruh 980 pasangan data digunakan dalam training.

```python
data_cosinus = np.cos(np.linspace(0, 100, 1000))
X_cos, y_cos = create_sequences(data_cosinus, seq_length)
X_cos = torch.FloatTensor(X_cos).unsqueeze(-1)
y_cos = torch.FloatTensor(y_cos).unsqueeze(-1)

plt.figure(figsize=(10, 4))
plt.plot(data_cosinus, label='Cosinus')
plt.xlabel('Urutan Data')
plt.ylabel('Nilai')
plt.title('Dataset Cosinus')
plt.legend()
plt.show()
```

Gambar 3.12 Screenshot kode dataset cosinus



Gambar 3.13 Output visualisasi data cosinus

3.4.2 Konfigurasi dan Pelatihan Model

Tiga konfigurasi yang digunakan adalah hidden size 32 dengan satu layer, hidden size 64 dengan satu layer, serta hidden size 64 dengan dua layer. Setiap konfigurasi dijalankan pada GRU dan LSTM menggunakan fungsi training yang sama. Nama model diberi akhiran 1, 2, dan 3 untuk membedakan konfigurasi.

```python
konfigurasi = [(32, 1), (64, 1), (64, 2)]
hasil_tugas = []
detail_tugas = {}
for nomor, (hidden_size, num_layers) in enumerate(konfigurasi, start=1):
    for arsitektur, kelas in [("GRU", GRUModel), ("LSTM", LSTMModel)]:
        nama = f"{arsitektur}_{nomor}"
        print(f"Training {nama}: hidden={hidden_size}, layers={num_layers}")
        model_t, losses_t, prediksi_t, waktu_t = jalankan_eksperimen(
            kelas, X_cos, y_cos, hidden_size, num_layers
        )
        hasil_tugas.append({
            "Model": nama, "Arsitektur": arsitektur,
            "Hidden Size": hidden_size, "Num Layers": num_layers,
            "Loss Akhir": losses_t[-1], "Waktu Training (s)": waktu_t,
            "Catatan": f"{num_layers} layer, hidden size {hidden_size}"
        })
        detail_tugas[nama] = {"losses": losses_t, "predictions": prediksi_t}

tabel_hasil = pd.DataFrame(hasil_tugas)
kolom_modul = ["Model", "Hidden Size", "Num Layers", "Loss Akhir", "Waktu Training (s)", "Catatan"]
print("Tabel eksperimen GRU sesuai kolom modul")
display(tabel_hasil.loc[tabel_hasil["Arsitektur"] == "GRU", kolom_modul])
print("Tabel perbandingan dengan LSTM")
display(tabel_hasil[kolom_modul])
```

Gambar 3.14 Screenshot kode eksperimen GRU dan LSTM

3.4.3 Penyusunan Tabel Hasil

Loss akhir dan waktu training disimpan bersama nama model, hidden size, jumlah layer, dan catatan konfigurasi. Data tersebut diubah menjadi DataFrame. Tabel GRU ditampilkan lebih dahulu sesuai format modul, lalu diikuti tabel perbandingan dengan LSTM.

```
Training GRU_1: hidden=32, layers=1
Training LSTM_1: hidden=32, layers=1
Training GRU_2: hidden=64, layers=1
Training LSTM_2: hidden=64, layers=1
Training GRU_3: hidden=64, layers=2
Training LSTM_3: hidden=64, layers=2
Tabel eksperimen GRU sesuai kolom modul
```

	Model	Hidden Size	Num Layers	Loss Akhir	Waktu Training (s)	Catatan
0	GRU_1	32	1	0.000118	1.730997	1 layer, hidden size 32
2	GRU_2	64	1	0.000097	3.931472	1 layer, hidden size 64
4	GRU_3	64	2	0.000751	11.717173	2 layer, hidden size 64

```
Tabel perbandingan dengan LSTM
```

	Model	Hidden Size	Num Layers	Loss Akhir	Waktu Training (s)	Catatan
0	GRU_1	32	1	0.000118	1.730997	1 layer, hidden size 32
1	LSTM_1	32	1	0.000505	2.947198	1 layer, hidden size 32
2	GRU_2	64	1	0.000097	3.931472	1 layer, hidden size 64
3	LSTM_2	64	1	0.000325	7.953191	1 layer, hidden size 64
4	GRU_3	64	2	0.000751	11.717173	2 layer, hidden size 64
5	LSTM_3	64	2	0.001233	14.798735	2 layer, hidden size 64

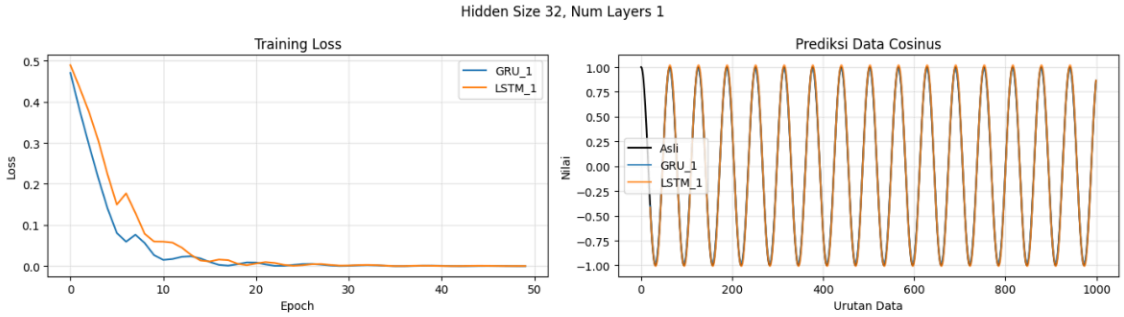
Gambar 3.15 Screenshot output tabel hasil eksperimen

3.4.4 Visualisasi Perbandingan

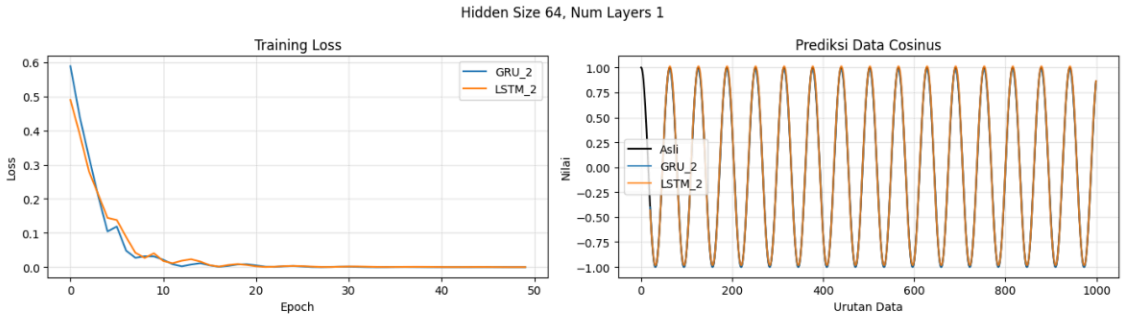
Setiap konfigurasi ditampilkan dalam dua grafik. Grafik kiri membandingkan loss selama training, sedangkan grafik kanan membandingkan prediksi dengan data cosinus asli. Urutan indeks prediksi tetap dimulai dari 20 seperti pada praktikum sinus.

```python
for nomor, (hidden_size, num_layers) in enumerate(konfigurasi, start=1):
    fig, axes = plt.subplots(1, 2, figsize=(14, 4))
    axes[1].plot(data_cosinus, label="Asli", color="black")
    for arsitektur in ["GRU", "LSTM"]:
        nama = f"{arsitektur}_{nomor}"
        info = detail_tugas[nama]
        axes[0].plot(info["losses"], label=nama)
        axes[1].plot(
            np.arange(seq_length, len(info["predictions"]) + seq_length),
            info["predictions"].ravel(), label=nama, alpha=0.8
        )
    axes[0].set(title="Training Loss", xlabel="Epoch", ylabel="Loss")
    axes[1].set(title="Prediksi Data Cosinus", xlabel="Urutan Data", ylabel="Nilai")
    for ax in axes:
        ax.legend()
        ax.grid(alpha=0.3)
    fig.suptitle(f"Hidden Size {hidden_size}, Num Layers {num_layers}")
    plt.tight_layout()
    plt.show()
```

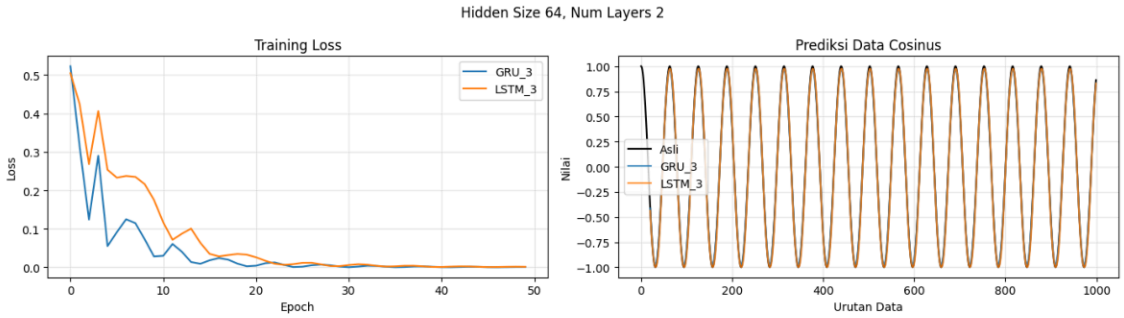
Gambar 3.16 Screenshot kode grafik perbandingan



Gambar 3.17 Output perbandingan hidden size 32 dan satu layer



Gambar 3.18 Output perbandingan hidden size 64 dan satu layer



Gambar 3.19 Output perbandingan hidden size 64 dan dua layer

3.5 Pemeriksaan Pembentukan Data

Pembentukan data dapat diperiksa dari jumlah sampel dan posisi targetnya. Sebanyak 1.000 titik dikurangi panjang sequence 20 menghasilkan 980 pasangan. Tidak ada padding atau penambahan nilai di awal deret. Sampel terakhir masih menggunakan data asli sampai indeks 998 dan targetnya adalah titik terakhir pada indeks 999.

Pada fungsi cosinus, titik pertama bernilai 1 karena cos(0) = 1. Jarak antartitik pada rentang 0 sampai 100 adalah 100/999. Target sampel pertama berada pada nilai cos(20 x 100/999), sekitar -0,4180. Perhitungan ini dapat dipakai untuk memeriksa bahwa target memang satu langkah setelah input, bukan nilai terakhir yang masih termasuk input.

Pada data sinus, posisi indeksnya sama tetapi nilai pembangkitnya berbeda. Nilai target pertama adalah sin(20 x 100/999), sekitar 0,9085. Contoh tersebut merupakan perhitungan langsung dari dataset, bukan keluaran prediksi model. Perbedaan fase sinus dan cosinus membuat nilai awalnya berbeda meskipun bentuk gelombangnya sama.

Data input dan target diubah menjadi FloatTensor agar sesuai dengan parameter model PyTorch. Konversi ini membuat nilainya menggunakan presisi float32. Pembulatan kecil dapat terjadi, tetapi tidak mengubah jumlah sampel, urutan waktu, atau hubungan antara input dan target.

Karena tidak ada pembagian data, semua pasangan yang digunakan untuk menghasilkan prediksi juga dipakai pada pelatihan. Pemeriksaan bentuk tensor memastikan program menerima data dengan susunan yang benar, tetapi tidak menggantikan pengujian pada data baru.

Pemeriksaan	Data sinus	Data cosinus
Jumlah titik	1.000	1.000
Jumlah sequence	980	980
Panjang input	20	20
Indeks target awal	20	20
Target awal sekitar	0,9085	-0,4180

Tabel 3.1 Pemeriksaan susunan dataset

3.6 Urutan Pembaruan Bobot dan Pencatatan Loss

Loop training memakai seluruh 980 sampel dalam satu batch. Satu epoch berarti seluruh pasangan input dan target diproses satu kali. Karena tidak ada mini-batch, optimizer memperbarui bobot satu kali pada setiap epoch. Selama 50 epoch, terdapat 50 langkah pembaruan bobot.

optimizer.zero_grad() diperlukan karena gradien pada PyTorch secara bawaan diakumulasikan. Setelah gradien lama dikosongkan, outputs = model(X) menghasilkan prediksi berdasarkan bobot saat itu. criterion(outputs, y) menghitung rata-rata kuadrat selisih seluruh prediksi dengan target.

loss.backward() menghitung gradien dari loss terhadap parameter. Pada jaringan recurrent, pengaruh parameter ditelusuri melalui langkah-langkah sequence. optimizer.step() kemudian menggunakan gradien tersebut untuk memperbarui bobot. Tahapan ini berlangsung berulang dengan data yang sama.

Walaupun losses.append(loss.item()) ditulis setelah optimizer.step(), nilai loss tersebut sudah dihitung sebelum bobot diperbarui. Karena itu, loss pada epoch ke-50 tidak persis sama dengan MSE yang akan diperoleh jika prediksi dihitung ulang setelah pembaruan terakhir. Grafik prediksi pada notebook menggunakan bobot setelah pembaruan terakhir.

Perbedaan waktu perhitungan tersebut perlu diperhatikan saat membaca tabel dan grafik. Tabel menggunakan loss terakhir yang dicatat oleh loop, sedangkan garis prediksi berasal dari pemanggilan model sesudah training. Keduanya tetap sesuai dengan urutan kode modul dan tidak perlu dipaksa menjadi angka yang sama.

Urutan	Operasi	Fungsi
1	zero_grad	Mengosongkan gradien
2	model(X)	Menghitung prediksi
3	MSELoss	Menghitung loss
4	backward	Menghitung gradien
5	step	Memperbarui bobot

Tabel 3.2 Urutan operasi dalam satu epoch

BAB IV
HASIL DAN PEMBAHASAN

4.1 Hasil Praktikum Deret Sinus

Model	Hidden Size	Layer	Loss Akhir	Waktu (s)
GRU	64	1	0,000070	3,692325
LSTM	64	1	0,000403	7,330393

Tabel 4.1 Hasil perbandingan GRU dan LSTM pada data sinus

Pada pengukuran ulang data sinus, GRU memperoleh loss akhir 0,000070, sedangkan LSTM memperoleh 0,000403. Waktu training GRU adalah 3,692325 detik dan LSTM 7,330393 detik. Dengan pengaturan ini, GRU memiliki loss lebih rendah dan menyelesaikan training lebih cepat.

Loss GRU pada tabel berbeda dari output training awal yang menampilkan 0,0003. Keduanya berasal dari pelatihan model yang berbeda. Training awal mengikuti kode modul tanpa penetapan seed, sedangkan percobaan pembanding membuat model baru dengan seed 42. Karena itu, hasil pengukuran ulang tidak digunakan untuk mengganti angka pada output training awal.

4.2 Hasil Tugas Praktikum Deret Cosinus

Model	Hidden Size	Layer	Loss Akhir	Waktu (s)
GRU_1	32	1	0,000118	1,730997
GRU_2	64	1	0,000097	3,931472
GRU_3	64	2	0,000751	11,717173

Tabel 4.2 Hasil tiga konfigurasi GRU pada data cosinus

GRU_2 menghasilkan loss paling kecil, yaitu 0,000097. GRU_1 memiliki loss sedikit lebih besar, yaitu 0,000118, tetapi waktu trainingnya paling singkat. GRU_3 membutuhkan waktu paling lama dan memiliki loss tertinggi di antara ketiga konfigurasi GRU.

4.2.1 Pengaruh Hidden Size

Penambahan hidden size dari 32 menjadi 64 pada model satu layer menurunkan loss dari 0,000118 menjadi 0,000097. Selisihnya hanya 0,000021, sedangkan waktu training bertambah dari 1,730997 menjadi 3,931472 detik. Dengan demikian, GRU_2 memberikan kecocokan yang lebih baik pada data training, tetapi GRU_1 lebih hemat waktu dengan loss yang masih rendah.

4.2.2 Pengaruh Penambahan Layer

GRU_2 dan GRU_3 sama-sama menggunakan hidden size 64. Ketika layer ditambah menjadi dua, loss meningkat dari 0,000097 menjadi 0,000751 dan waktu training bertambah sekitar 7,786 detik. Grafik juga menunjukkan perubahan loss yang lebih besar pada awal training model dua layer. Pada 50 epoch dan learning rate 0,01, penambahan kedalaman belum memberi perbaikan.

Hasil ini tidak cukup untuk menyatakan model dua layer selalu lebih buruk. Model yang lebih dalam mungkin membutuhkan pengaturan optimasi yang berbeda. Penambahan epoch atau penurunan learning rate dapat dicoba, tetapi keduanya belum diuji pada praktikum ini.

4.3 Perbandingan GRU dan LSTM

Model	Hidden Size	Layer	Loss Akhir	Waktu (s)
GRU_1	32	1	0,000118	1,730997
LSTM_1	32	1	0,000505	2,947198
GRU_2	64	1	0,000097	3,931472
LSTM_2	64	1	0,000325	7,953191
GRU_3	64	2	0,000751	11,717173
LSTM_3	64	2	0,001233	14,798735

Tabel 4.3 Perbandingan GRU dan LSTM pada data cosinus

GRU memperoleh loss lebih rendah daripada LSTM pada ketiga konfigurasi. Selisih waktunya paling besar pada hidden size 64 dan satu layer, yaitu sekitar 4,022 detik. Pada konfigurasi ini, GRU selesai dalam 3,931472 detik, sedangkan LSTM membutuhkan 7,953191 detik.

Hasil tersebut mendukung penggunaan GRU untuk data periodik sederhana pada percobaan ini. Namun, masing-masing konfigurasi hanya dijalankan sekali dengan satu seed. Waktu juga dapat dipengaruhi beban CPU dan urutan pelatihan. Pengukuran berulang diperlukan jika ingin membandingkan efisiensi secara lebih pasti.

Angka LSTM pada laporan Praktikum 4 tidak dibandingkan langsung dengan hasil sekarang. Laporan sebelumnya menggunakan pembagian training dan testing serta pengaturan perangkat yang berbeda. Pada praktikum ini, arsitektur LSTM dilatih ulang dengan seluruh sequence yang sama seperti GRU sehingga dasar perbandingannya lebih sesuai.

4.4 Analisis Kurva Loss dan Prediksi

Pada konfigurasi satu layer, kurva loss turun cepat di awal training, sempat berfluktuasi, kemudian mendekati nol. Pada dua layer, fluktuasi awal terlihat lebih besar. Walaupun demikian, prediksi kedua arsitektur tetap mengikuti bentuk gelombang cosinus dan sebagian besar garisnya tampak berimpit dengan data asli.

Kesesuaian grafik perlu dibaca bersama tabel loss karena perbedaan kecil sulit terlihat pada skala grafik. Selain itu, semua input berasal dari data yang digunakan saat training. Percobaan ini belum menguji generalisasi dan belum dapat memastikan ada atau tidaknya overfitting. Tidak ada regularisasi atau dropout tambahan yang diterapkan.

4.5 Jawaban Pertanyaan Reflektif

4.5.1 Mengapa GRU Lebih Cepat daripada LSTM

GRU memiliki dua gerbang utama dan tidak menggunakan cell state terpisah. Pada konfigurasi yang sama, jumlah parameternya lebih sedikit daripada LSTM sehingga komputasinya umumnya lebih ringan. Dalam percobaan ini, waktu GRU memang lebih singkat pada semua pasangan konfigurasi. Hasil tersebut tetap terbatas pada perangkat dan pengaturan yang digunakan.

4.5.2 Peran Update Gate dalam GRU

Update gate mengatur seberapa banyak informasi lama dipertahankan saat hidden state diperbarui. Gerbang ini menyeimbangkan penggunaan hidden state sebelumnya dan kandidat informasi baru. Dengan cara tersebut, informasi yang masih berguna dapat dibawa ke langkah berikutnya.

4.5.3 Penggunaan GRU untuk Teks yang Sangat Panjang

GRU dapat digunakan untuk teks karena memproses data secara berurutan. Namun, teks yang sangat panjang tetap sulit ditangani karena informasi harus diringkas ke dalam hidden state yang ukurannya terbatas. Hubungan antarkata yang berjauhan dapat menjadi lebih sulit dipelajari. Kecocokan GRU perlu dinilai berdasarkan tugas dan panjang teks; hasil pada sequence sepanjang 20 dalam praktikum ini belum mewakili kondisi teks yang sangat panjang.

4.6 Selisih Waktu Training Antarmodel

Perbandingan waktu dibaca per pasangan konfigurasi. Pada hidden size 32 dan satu layer, GRU membutuhkan 1,730997 detik, sedangkan LSTM membutuhkan 2,947198 detik. Selisihnya sekitar 1,216 detik. Jika waktu LSTM digunakan sebagai pembanding, waktu GRU lebih singkat sekitar 41,27 persen.

Pada hidden size 64 dan satu layer, selisih waktu menjadi sekitar 4,022 detik. GRU membutuhkan 3,931472 detik dibandingkan LSTM 7,953191 detik. Pengurangan waktunya sekitar 50,57 persen. Ini merupakan selisih relatif terbesar di antara ketiga pasangan pada percobaan cosinus.

Pada dua layer, GRU tetap lebih cepat, tetapi keuntungan relatifnya lebih kecil. GRU membutuhkan 11,717173 detik, sedangkan LSTM 14,798735 detik. Selisihnya sekitar 3,082 detik atau 20,82 persen dari waktu LSTM. Jadi, keuntungan waktu GRU tidak memiliki persentase yang tetap ketika konfigurasi diubah.

Persentase tersebut dihitung dengan rumus (waktu LSTM - waktu GRU) / waktu LSTM x 100 persen. Ini adalah pengolahan ulang angka pada tabel hasil, bukan pengukuran baru. Karena training hanya dilakukan satu kali per konfigurasi, angka persentasenya belum menggambarkan rata-rata dari beberapa pengulangan.

Hasil ini juga menunjukkan bahwa dua model dengan perbedaan parameter tertentu tidak selalu memiliki perbandingan waktu yang sama. Jumlah operasi bertambah ketika layer ditambahkan, tetapi waktu aktual tetap dipengaruhi cara operasi tersebut dijalankan di CPU.

Konfigurasi	Selisih waktu sekitar	Pengurangan waktu GRU
Hidden 32, 1 layer	1,216 detik	41,27%
Hidden 64, 1 layer	4,022 detik	50,57%
Hidden 64, 2 layer	3,082 detik	20,82%

Tabel 4.4 Selisih waktu GRU terhadap LSTM

4.7 Pemilihan Konfigurasi Berdasarkan Hasil

Pemilihan konfigurasi bergantung pada hasil yang ingin diprioritaskan. Jika hanya melihat loss training paling kecil, GRU_2 menjadi pilihan pada percobaan cosinus. Namun, jika waktu training juga dipertimbangkan, GRU_1 tetap layak karena lossnya hanya berbeda 0,000021 dari GRU_2 dan waktunya lebih singkat sekitar 2,200 detik.

GRU_1 memiliki 3.393 parameter, sedangkan GRU_2 memiliki 12.929 parameter. Hidden size yang meningkat dua kali tidak membuat parameter hanya meningkat dua kali karena terdapat matriks bobot hidden yang ukurannya bergantung pada kuadrat hidden size. Kenaikan kapasitas ini perlu dibandingkan dengan perbaikan loss yang diperoleh.

Pada GRU_3, parameter bertambah menjadi 37.889. Layer kedua menerima masukan berukuran 64 dari layer pertama, bukan satu fitur seperti pada layer awal. Itulah sebabnya jumlah parameter meningkat cukup besar. Pada hasil aktual, tambahan parameter ini belum diikuti penurunan loss dalam 50 epoch.

Hal serupa terjadi pada LSTM. Konfigurasi hidden size 64 dan satu layer menghasilkan loss 0,000325, lebih kecil daripada hidden size 32 yang sebesar 0,000505. Ketika layer ditambah menjadi dua, loss justru naik menjadi 0,001233. Dengan demikian, pola bahwa model lebih dalam belum memberi keuntungan terlihat pada kedua arsitektur.

Kesimpulan praktisnya adalah satu layer sudah memadai untuk mempelajari pola pada data ini dengan pengaturan yang digunakan. Pernyataan tersebut tidak berarti konfigurasi satu layer paling baik untuk semua masalah. Data yang lebih kompleks, sequence lebih panjang, atau jumlah epoch berbeda dapat menghasilkan urutan kinerja yang berbeda.

Pertimbangan	Konfigurasi GRU	Dasar hasil
Loss terendah	Hidden 64, 1 layer	Loss 0,000097
Training tercepat	Hidden 32, 1 layer	Waktu 1,730997 s
Dua layer	Hidden 64, 2 layer	Belum memperbaiki loss

Tabel 4.5 Pertimbangan konfigurasi GRU

BAB V
PENUTUP

5.1 Kesimpulan

GRU berhasil digunakan untuk mempelajari pola sinus dan cosinus dengan input sepanjang 20 nilai. Pada training awal data sinus, loss yang ditampilkan turun dari 0,0326 pada epoch ke-10 menjadi 0,0003 pada epoch ke-50. Pada pengukuran ulang dengan seed 42, GRU memperoleh loss 0,000070 dan waktu training 3,692325 detik, lebih rendah dan lebih singkat daripada LSTM pada konfigurasi yang sama.

Pada data cosinus, GRU_2 dengan hidden size 64 dan satu layer menghasilkan loss terendah sebesar 0,000097. GRU_1 menjadi konfigurasi tercepat dengan waktu 1,730997 detik dan loss 0,000118. Penambahan menjadi dua layer pada GRU_3 meningkatkan loss menjadi 0,000751 serta memperpanjang waktu training menjadi 11,717173 detik. Untuk pengaturan percobaan ini, satu layer sudah memberikan hasil yang lebih baik.

GRU memiliki loss lebih rendah dan waktu training lebih singkat daripada LSTM pada ketiga konfigurasi cosinus. Kesimpulan ini berlaku pada data yang digunakan saat training. Kemampuan model pada data baru belum diukur karena praktikum mengikuti kode modul yang memakai seluruh sequence untuk training dan prediksi.

DAFTAR PUSTAKA

[1] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. 2014 Conf. Empirical Methods Natural Language Processing (EMNLP)*, Doha, Qatar, 2014, pp. 1724-1734, doi: 10.3115/v1/D14-1179.

[2] PyTorch, "GRU," *PyTorch Documentation*. Accessed: Oct. 28, 2026. [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.GRU.html

[3] PyTorch, "LSTM," *PyTorch Documentation*. Accessed: Oct. 28, 2026. [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, San Diego, CA, USA, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980